Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt zespołowy – laboratorium 2

Implementacja bazy danych szeregów czasowych w języku Rust

Architektura systemu

Autorzy: Hubert Błonowski
Jan Zaborski

Promotor: mgr inż. Paweł Sobótka

Wersja dokumentu: 1.0
Data: 3 listopada 2026 r.

`https://github.com/janzaborski/rust-tsdb`

Streszczenie

Dokument przedstawia projekt jednowęzłowej bazy danych szeregów czasowych: podział na moduły, model danych, zapis i odczyt, przechowywanie na dysku oraz odzyskiwanie po awarii. Opisuje wykonywanie zapytań, komunikację HTTP, integrację z Grafaną i granice dostępu do systemu. Jest specyfikacją docelową. Parametry wydajnościowe są celami do weryfikacji.

Historia zmian

Wersja	Data	Autorzy	Zakres
1.0	03.11.2026	H. Błonowski, J. Zaborski	Napisanie dokumentu architektury systemu.

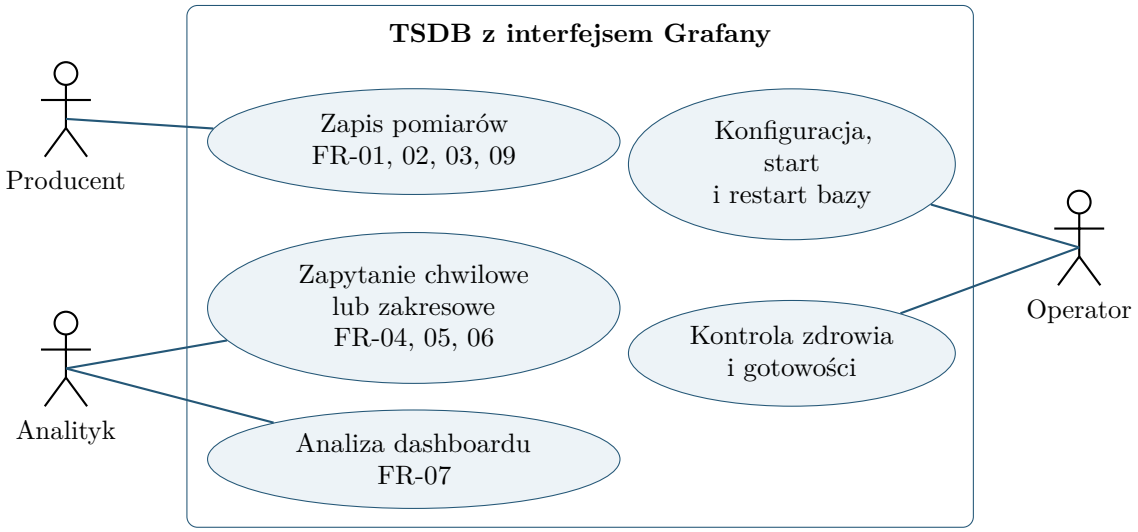
Spis treści

1 Cel, zakres i główne decyzje . . . 1
2 Architektura i odpowiedzialności modułów . . . 1
3 Model danych i współbieżność . . . 2
4 Ścieżka zapisu i architektura WAL . . . 4
5 Checkpoint, pliki i odzyskiwanie . . . 5
6 Zapytania i komunikacja HTTP . . . 6
7 Technologie, wdrożenie i interfejs użytkownika . . . 8

1 Cel, zakres i główne decyzje

Celem jest własny silnik do przyjmowania pomiarów, przechowywania historii i wykonywania agregacji czasowych. Użytkownik analizuje dane w Grafanie. Zakres obejmuje HTTP, etykiety, pięć agregacji, out-of-order w otwartym buforze oraz wdrożenie w Docker Compose.

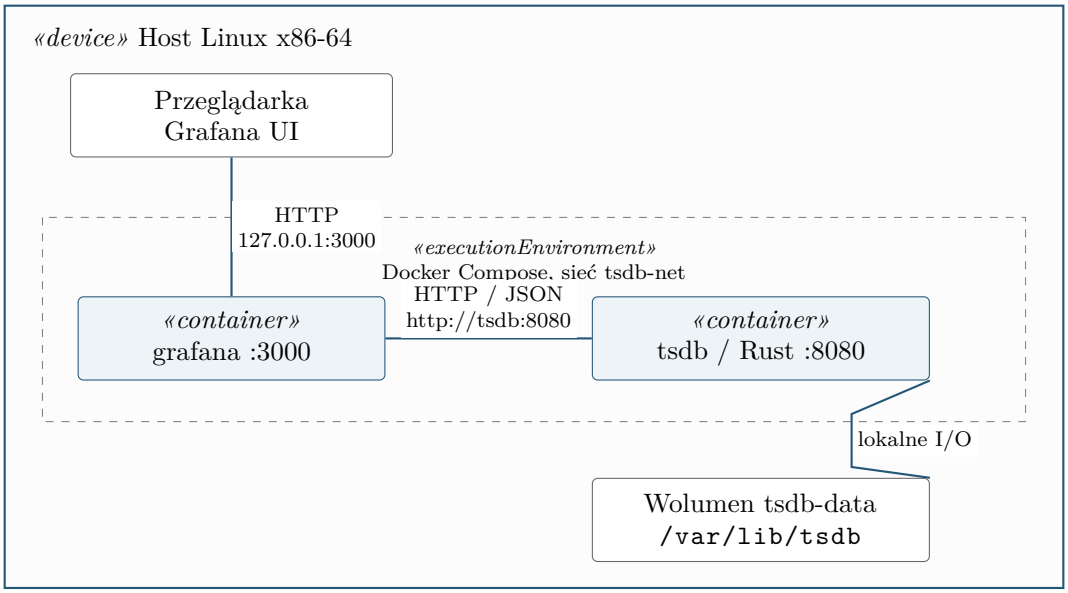
Bloki aktywne i zamrożone umożliwiają przyjmowanie pomiarów podczas zapisu segmentów. Poza zakresem pierwszej wersji pozostają klaster, replikacja, kompresja, retencja, kompaktowanie, alertowanie i własny frontend. Backup wymaga zatrzymania bazy i skopiowania katalogu danych.



Rysunek 1: Diagram przypadków użycia UML.

2 Architektura i odpowiedzialności modułów

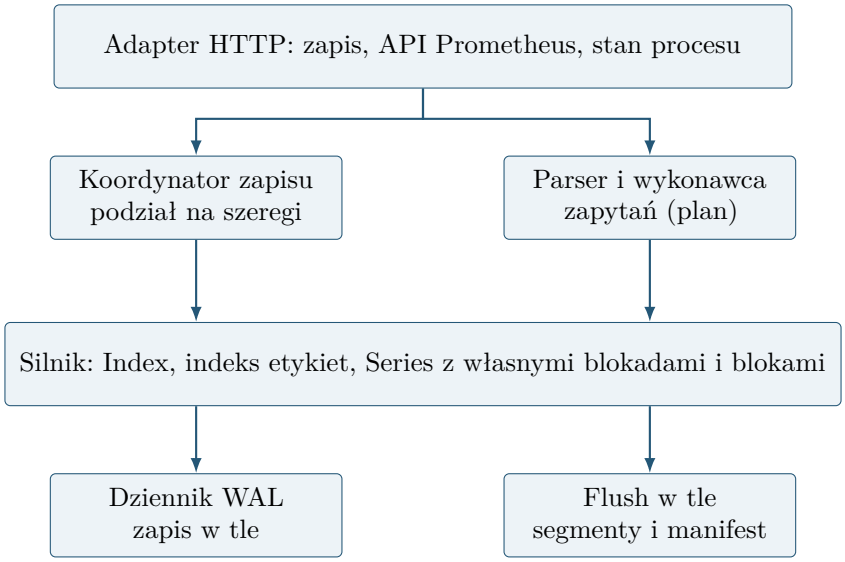
Przeglądarka komunikuje się z Grafaną. Jej serwer odpytuje TSDB. Pomiary są przyjmowane przez API HTTP. Silnik korzysta z lokalnego systemu plików.



Rysunek 2: Wdrożenie na jednym hoście.

Adapter HTTP zna protokół, parser zna składnię, a silnik zna dane i pliki. Dzięki temu magazyn

można rozwijać/testować bez HTTP, a wykonawcę zapytań z pamięciowym interfejsem odczytu.



Rysunek 3: Zależności logiczne.

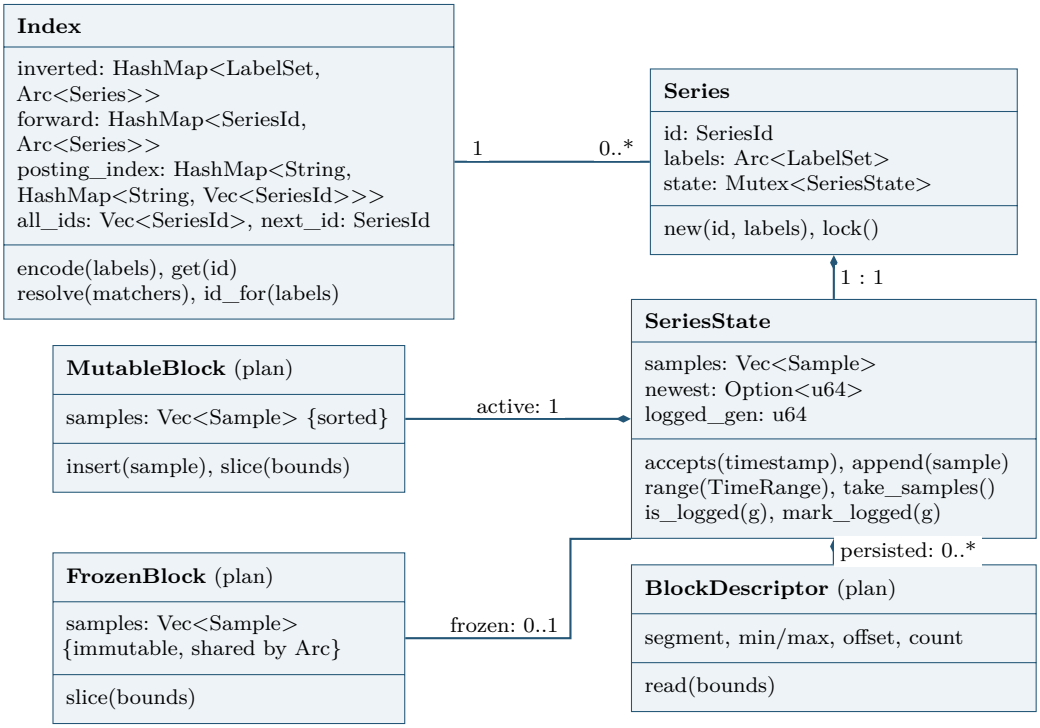
Moduł	Odpowiedzialność i kontrakt
HTTP i walidacja	Dekodowanie, kanonizacja, limity i błędy, wywołanie zapisu lub zapytania.
Koordynator zapisu	Podział pracy według szeregów, walidacja punktów i raportowanie częściowego wyniku.
Index i Series	Klucz ↔ Series, dobór po etykietach, aktywny bufor, bloki zamrożone i segmenty dyskowe.
WAL	Dziennik operacji, potwierdzanie zapisu i odtwarzanie po awarii.
Magazyn segmentów	Niezmienne pliki, katalog danych i odzyskiwanie po awarii.
Parser i wykonawca	Analiza składni, plan, odczyt i obliczenia, przygotowanie wyniku zapytania.

3 Model danych i współbieżność

`Index` jest katalogiem szeregów i indeksem etykiet, a `Series` reprezentuje pojedynczy szereg. Są to nazwy używane w kodzie. Klucz szeregu to `LabelSet`, a pomiar to `Sample`. Model obejmuje bufor aktywny, niezmienne bloki oczekujące na zapis i segmenty dyskowe. Równoległy odczyt oraz obsługa zamrożonych bloków opisują stan docelowy

Szereg identyfikuje nazwa metryki i kanonicznie posortowane etykiety. Wewnętrzny LabelSet zawiera także `__name__`. Punkt to czas Unix w milisekundach (u64) i skończona wartość f64. Para zajmuje 16 B. SeriesId jest lokalny dla head/WAL i może zmienić się po restarcie. Segmenty i manifest identyfikują szeregi przez klucze.

Aktywny bufor utrzymuje pomiary w kolejności czasu. Opóźniony pomiar wymaga wstawienia we właściwe miejsce. Zamrożenie przekazuje bufor do zapisu bez pełnej kopii. Nowe pomiary trafiają do kolejnego bufora. Widok zapytania kopiuje potrzebne dane aktywne i współdzieli niezmienne bloki.



Rysunek 4: Model head.

3.1 Współbieżność i widoczność danych

Paczka jest jednostką transportową, a nie transakcją. Jej punkty mogą być publikowane i odrzucane niezależnie. Błąd jednego pomiaru nie wycofuje wcześniejszych zapisów, a odczyt może zobaczyć część przetwarzanej paczki. Nie zapewnia się izolacji ani wspólnej migawki czasu dla wielu szeregów.

Koordynator rozdziela pracę według szeregów. Pracownik utrzymuje blokadę tylko jednego szeregu i przetwarza małą porcję pomiarów, następnie ją zwalnia. Nie zbiera blokad wszystkich szeregów paczki. Rejestracja nowego klucza w katalogu jest krótka. Kodowanie, odczyt plików i oczekiwanie na potwierdzenie dziennika nie blokują katalogu.

Zapytanie utrwala dane osobno dla każdego wybranego szeregu i zwalnia jego blokadę przed przejściem do następnego. Poszczególne szeregi mogą odpowiadać różnym chwilom odczytu. Szeregi utworzone w trakcie zapytania mogą być pominięte. Jednoczesny zrzut na dysk nie może powodować podwójnego zwracania ani pomijania już wybranych próbek.

Pozostają gwarancje poprawnego pojedynczego pomiaru, chronologicznego odczytu i odzyskiwania potwierdzonych zapisów. Zmniejszenie zakresu blokad ogranicza wzajemne oczekiwanie. Koszt wspólnego dziennika i obciążonych szeregów nadal wymaga pomiaru.

3.2 Duplikaty i opóźnione punkty

Identyczny punkt w aktywnym buforze jest duplikatem bez zmian. Inna wartość dla tego samego czasu odrzuca tylko ten punkt jako konflikt. Nie modyfikuje się zamkniętych zakresów, także dla identycznego duplikatu. Dla nowego punktu obowiązuje

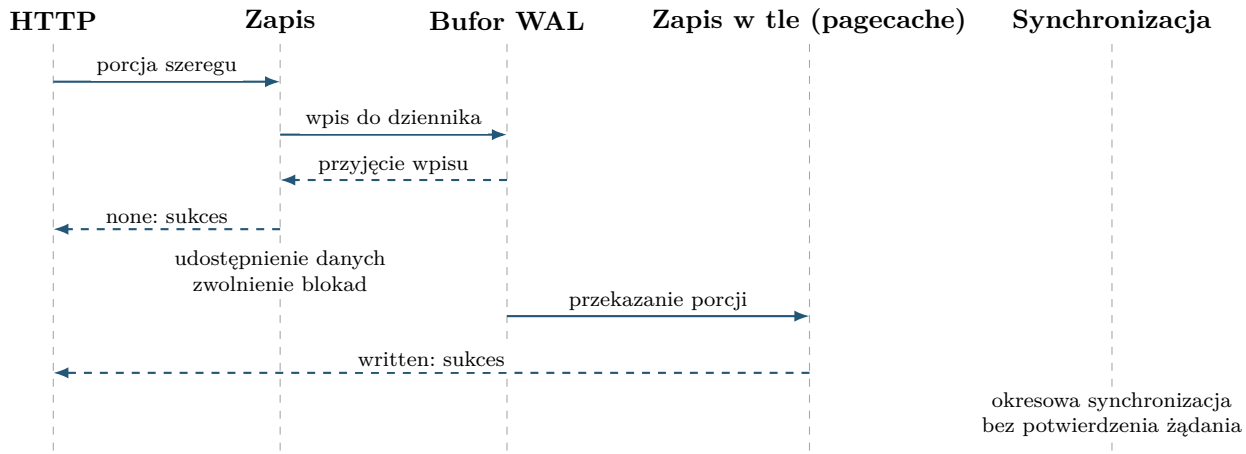
$$t \geq H_s - W,$$

gdzie H_s to największy przyjęty czas, a W okno opóźnienia (domyślnie 5 min). Każdy punkt porównuje się ze stanem szeregu w chwili jego przetwarzania. W obrębie pracy dla jednego szeregu pomiary porządkuje się po czasie. Równe czasy zachowują kolejność wejścia.

4 Ścieżka zapisu i architektura WAL

WAL (Write-Ahead Log) pozwala odzyskać pomiary, które nie trafiły jeszcze do segmentów danych. Po sprawdzeniu składni żądania walidacja i rezerwacja pamięci odbywają się dla kolejnych punktów lub małych porcji jednego szeregu. Poprawne pomiary trafiają do dziennika i są udostępniane zapytaniom. Błędne są raportowane osobno.

Paczka HTTP może tworzyć wiele niezależnych wpisów dziennika, a pojedynczy wpis może łączyć kilka pomiarów dla wydajności. Granica wpisu nie ustanawia transakcji całego żądania. Awaria lub przerwanie przetwarzania może pozostawić tylko część pomiarów paczki. Utrata połączenia nie wycofuje już przyjętych danych.



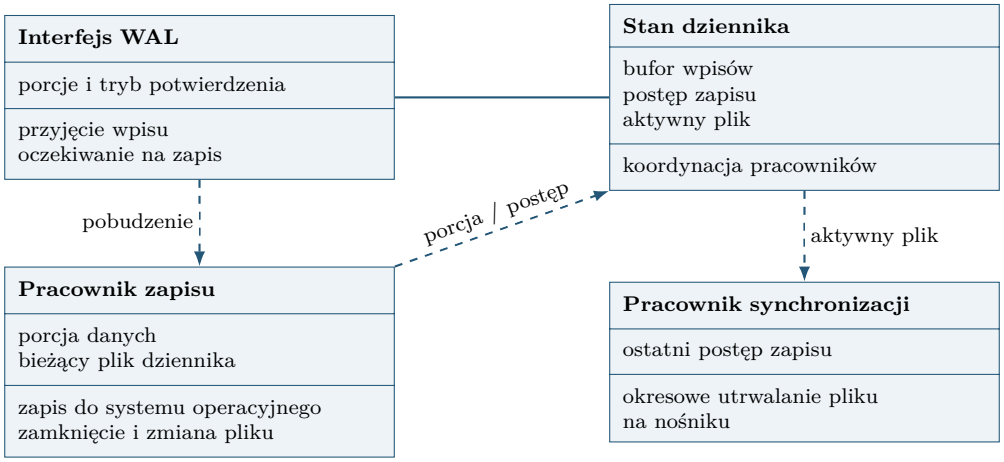
Rysunek 5: Przepływ zapisu małej porcji jednego szeregu. Paczka HTTP przechodzi ten proces wielokrotnie. Nie stanowi jednej transakcji.

Pracownik zapisu w tle przekazuje zebrane wpisy do pliku dziennika. W tym czasie kolejne żądania mogą korzystać z drugiego bufora. Osobny pracownik okresowo zleca synchronizację pliku z nośnikiem. Rozdzielenie tych zadań pozwala łączyć małe zapisy i ogranicza czas blokowania żądań.

Tryb	Warunek odpowiedzi po udostępnieniu danych do odczytu
`none` (domyślny)	Przyjęte punkty opublikowane w pamięci. Awaria procesu może spowodować ich utratę.
`written`	Wpisy przyjętych punktów przekazane do systemu operacyjnego. Przyjęte dane przetrwają awarię samego procesu przy sprawnym hoście i nośniku. Nie ma gwarancji przetrwania utraty zasilania.

Okresowa synchronizacja zmniejsza ryzyko utraty danych przy awarii hosta, ale nie jest osobnym potwierdzanym trybem żądania. Jej interwał nie wyznacza gwarantowanego okna utraty danych: zależy ono także od obciążenia i opóźnień urządzenia. Zapis do pamięci podręcznej systemu i trwałe utrwalenie na nośniku są odrębnymi operacjami.

Po osiągnięciu progu rozmiaru dziennik przechodzi do kolejnego pliku. Pojedynczy wpis nie jest dzielony między pliki. Przed checkpointem silnik ustala zamknięty, zsynchronizowany zbiór plików. Dopiero po trwałym opublikowaniu odpowiadających im segmentów można usunąć tę część dziennika. Po awarii odtwarzane są pełne wpisy nowsze od checkpointu. Błąd zapisu lub synchronizacji zatrzymuje dalsze przyjmowanie danych. Żądania bez osiągniętego warunku potwierdzenia nie otrzymują sukcesu.



Rysunek 6: Elementy dziennika i ich odpowiedzialności. Zapis oraz synchronizacja są oddzielnymi zadaniami wykonywanymi w tle.

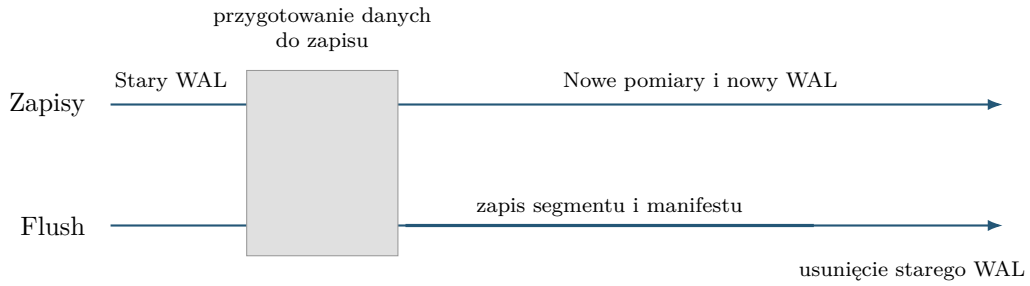
5 Checkpoint, pliki i odzyskiwanie

Pomiary są początkowo przechowywane w pamięci i dzienniku WAL. Co pewien czas baza zapisuje je do segmentów, czyli plików danych na dysku. Zapis jest uruchamiany po określonym czasie (domyślnie 30 minut) lub gdy bufor zbliża się do swojego limitu pamięci.

5.1 Zapis danych na dysk

1. Baza wydziela dane do zapisania. Nowe pomiary trafiają do kolejnego bufora, dzięki czemu mogą napływać podczas zapisu.
2. W tle powstaje segment danych. Po zakończeniu zapisu baza upewnia się, że plik został utrwalony na nośniku.
3. Baza aktualizuje manifest, czyli spis gotowych segmentów i informację o tym, które wpisy WAL są już w nich zapisane. Dopiero po utrwaleniu manifestu można zwolnić odpowiedni bufor i usunąć niepotrzebną część WAL.

Taki zakończony zapis tworzy checkpoint, czyli punkt, od którego baza może rozpocząć odtwarzanie nowszych danych po awarii. Przygotowanie danych wymaga krótkiej synchronizacji. Zapis plików odbywa się w tle. Pojedyncza paczka HTTP może zostać zapisana częściami w kolejnych segmentach.



Rysunek 7: Zapis danych na dysk odbywa się w tle, podczas przyjmowania nowych pomiarów.

5.2 Odzyskiwanie po awarii

Przy uruchomieniu baza odczytuje manifest i gotowe segmenty, a następnie uzupełnia je pomiarami z WAL, które nie zostały jeszcze zapisane w segmentach. Niekompletny wpis na końcu dziennika

może zostać pominięty. Uszkodzenie gotowych danych zatrzymuje uruchomienie, aby nie zwracać niepełnych wyników. Błąd zapisu na dysk zatrzymuje dalsze przyjmowanie pomiarów.

5.3 Buforowanie plików i kompaktowanie

Odczyt i zapis plików korzystają z pamięci podręcznej systemu operacyjnego (page cache). Każdy zrzut tworzy nowy segment.

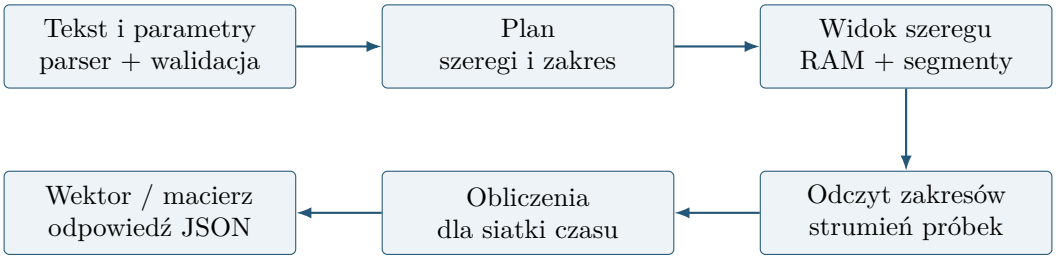
6 Zapytania i komunikacja HTTP

Istniejące endpointy to `POST/api/v1/write_json` i `GET/api/v1/read`. Obecny zapis przyjmuje tablicę obiektów `labels/samples`, z polami `t/v` dla pomiaru. Poniższy kontrakt zapisu jest docelowym rozszerzeniem tego formatu.

Obsługiwany podzbiór PromQL obejmuje nazwany selektor z dokładnym dopasowaniem etykiet, pięć funkcji czasowych i prosty skalar (np. 1+1 do próby połączenia). Bez regexów, zagnieżdżeń i agregacji między szeregami.

```
temperature_celsius{service="api"}
avg_over_time(temperature_celsius{service="api"}[5m])
```

Pozostałe funkcje: `min_over_time`, `max_over_time`, `sum_over_time`, `count_over_time`. Selektor wybiera ostatnią próbkę z $[T - D, T]$ (lookback $D = 5$ min). Funkcja działa osobno dla Series na $(T - R, T]$. Puste okno pomija Series, również dla count. Range powtarza obliczenie dla $T_k = start + k \cdot step \leq end$, korzystając z widoków poszczególnych szeregów. Indeks etykiet wybiera Series, indeksy czasowe ograniczają odczyt, a agregacje korzystają z przesuwanego okna.



Rysunek 8: Przepływ zapytania. Każdy etap może zakończyć się błędem.

6.1 Architektura wykonywania zapytań

Adapter HTTP sprawdza parametry. Parser przekształca tekst w drzewo wyrażenia, a planista ustala szeregi, zakres odczytu oraz siatkę czasu wyników. Indeks wybiera kandydatów według etykiet. Metadane segmentów pozwalają pominąć pliki spoza zakresu.

Zapytanie otrzymuje widok pamięci i segmentów osobno dla każdego szeregu, bez wspólnego punktu w czasie dla wszystkich szeregów. Odczyt może obejmować część równocześnie przetwarzanej paczki. Blokada pojedynczego szeregu jest zwalniana po skopiowaniu potrzebnych danych aktywnych i pobraniu uchwytów do bloków niezmiennych, przed odczytem dyskowym i obliczeniami. Warstwa odczytu dostarcza próbki chronologicznie, łącząc źródła bez podwójnego zwracania punktów podczas zrzutu na dysk.

Rozłączenie klienta sygnalizuje anulowanie wykonania. Pracownicy sprawdzają je między porcjami odczytu i obliczeń, a następnie zwalniają zasoby zapytania.

6.2 Interfejs HTTP

HTTP/1.1, UTF-8 i JSON. Zapytania także przez formularz POST. Wartości wyników są napisami, czasy wyników sekundami Unix. Zapis przyjmuje milisekundy i skończone f64.

Interfejs	Główny kontrakt
POST `/api/v1/write_json`	Jeden pomiar albo `points`, niezależne wyniki punktów.
GET/POST `/api/v1/query`	query, opcjonalny time, wynik chwilowy.
GET/POST `/api/v1/query_range`	query, start, end, dodatni step, macierz wyników na siatce czasu.
GET `/api/v1/labels`, `/api/v1/label/<name>/values`	Nazwy/wartości etykiet z całego katalogu.
GET `/api/v1/status/buildinfo`, `/healthz`, `/readyz`	Wersja TSDB, zdrowie i gotowość po recovery.

```
POST /api/v1/write_json
{"metric":"temperature_celsius","labels":{"service":"api"},
 "timestamp":1792843200000,"value":24.5}
```

Wynik zapisu: inserted, duplicates, rejected, errors. Zapytania stosują kopertę Prometheusa i vector/matrix/scalar. Błędy całego żądania: 400 składnia/koperta, 422 wykonanie zapytania, 503 przeciążenie/gotowość/timeout. Konflikty i odrzucenia pomiarów są elementami listy errors. Timeout nie wycofuje zapisu.

6.3 Wynik zapisu częściowego

Błędny JSON, niepoprawna koperta, zbyt duże żądanie lub brak gotowości stwierdzony przed rozpoczęciem przetwarzania odrzuca żądanie bez zapisu. Po dekodowaniu błędy poszczególnych pomiarów dotyczą tylko tych punktów. Konflikt wartości, zamknięty zakres, przekroczone okno opóźnienia albo brak zasobów nie wycofują wcześniej przyjętych punktów.

Zakończone przetwarzanie zwraca HTTP 200 z licznikami `inserted`, `duplicates`, `rejected` i listą `errors`. Każdy błąd wskazuje `point_index`, liczony od zera w oryginalnym wejściu, i `reason`. Sukces obsługi żądania nie oznacza przyjęcia wszystkich pomiarów. Również całkowite odrzucenie punktów jest rozpoznawane po licznikach. Po pełnym przetworzeniu suma trzech liczników równa się liczbie punktów.

Tryb `written` potwierdza przekazanie do systemu operacyjnego wszystkich przyjętych punktów tej odpowiedzi, także przy częściowych odrzuceniach. Nie tworzy transakcji i nie utrzymuje blokad szeregów. Ponowiony duplikat w otwartym buforze wymaga potwierdzenia jego wcześniejszego zapisu, jeśli klient wybrał ten tryb.

Timeout, rozłączenie lub awaria w trakcie obsługi może pozostawić częściowy, nieznany klientowi wynik. Błąd całego żądania nie dowodzi wtedy braku zapisów. Ponowienie może dać duplikaty albo odrzucenia zamkniętych zakresów. Nie gwarantuje się bezterminowej idempotencji.

7 Technologie, wdrożenie i interfejs użytkownika

Technologia	Uzasadnienie
Rust jako język programowania	Ergonomia, bezpieczeństwo pamięci i kontrola zasobów.
Axum, Tokio	Obsługa HTTP i współbieżnych żądań.
Serde, serde_json	Serializacja i odczyt danych JSON.
thiserror, tracing	Obsługa błędów i diagnostyka.
Grafana	Wizualizacja danych i zapytań.
Docker Compose, Linux	Powtarzalne wdrożenie na jednym hoście.

Wersje utrwala Cargo.lock, obraz Grafany jest przypięty. Platforma docelowa: Linux x86-64 i lokalny system plików. LOCK dopuszcza jeden proces na katalog. Usługi tsdb i grafana korzystają z sieci tsdb-net i wolumenu tsdb-data. Grafana odpytuje `http://tsdb:8080`. Przeglądarka korzysta z portu 3000. API na porcie 8080 i Grafana są publikowane na loopback. Baza działa bez uprawnień administratora. Profil dostępu opisano poniżej.

7.1 Uwierzytelnianie i zakres dostępu

Profil podstawowy obejmuje jeden zaufany host: API jest dostępne na loopback lub w wydzielonej sieci kontenerów. Silnik nie ma własnego systemu kont, haseł ani uprawnień do poszczególnych szeregów. Logowanie do Grafany chroni jej interfejs. Nie zabezpiecza samodzielnie bezpośredniego dostępu do API bazy.

Dostęp spoza zaufanego hosta wymaga osobnego profilu wdrożenia: reverse proxy przed API zapewnia HTTPS i uwierzytelnianie, a port silnika jest niedostępny bezpośrednio. Proponowany wariant minimalny to Basic Auth przez TLS, obsługiwany przez gotowe proxy, np. Nginx. Grafana potrafi przekazać dane Basic Auth i zweryfikować certyfikat serwera. Hasła i klucz prywatny są dostarczane jako sekrety poza repozytorium, a certyfikat serwera musi być zaufany przez klientów.

Jest to opcjonalne rozszerzenie konfiguracji wdrożenia, do zatwierdzenia przed udostępnieniem zdalnym. Własne zarządzanie użytkownikami, role oraz certyfikaty klienckie pozostają poza zakresem pierwszej wersji. Nie zakłada się, że samo dodanie hasła do Grafany zastępuje tę warstwę.

7.2 Przykładowa konfiguracja i budżety

Konfiguracja CLI jest wczytywana przy starcie. Compose przekazuje ją w command. Wartości domyślne:

Obszar	Wartości domyślne
Head i katalog	512 MiB active/frozen/Arc, metadata 64 MiB, 10 000 Series, buforowanie 30 min, out-of-order 5 min.
WAL i przyjęcie	32 MiB na dwa bufory dziennika, 64 MiB na ciała żądań, paczki i kodowanie, 8 przyjętych zapisów, 4 workers.
Paczka	Do 10 000 punktów, ciało 4 MiB, wpis WAL do 8 MiB.

Pamięć zamrożonych bloków jest rozliczana do zwolnienia ostatniego odczytu, także po publikacji segmentu. Zapas pojemności i zmiany rozmiaru buforów podlegają budżetom. Limity head nie są limitem RSS całego procesu. Logi obejmują identyfikator żądania, wynik, czas, tryb potwierdzenia, ilość oczekujących danych, zrzuty na dysk i raport odzyskiwania.

7.3 Interfejs graficzny

Interfejsem graficznym systemu będzie Grafana, dostępna z poziomu przeglądarki. Użytkownik będzie mógł przeglądać pomiary na wykresach, wybierać interesujące go szeregi i zakres czasu oraz porównywać dane z wynikami agregacji, takimi jak średnia, minimum i maksimum. Wykresy pozwolą obserwować trendy i zmiany wartości w czasie. Grafana będzie pobierać dane przez API bazy, a dashboardy umożliwią zestawienie kilku widoków danych na jednym ekranie.